JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Locale.Builder

java.lang.Object
    java.util.Locale.Builder

Enclosing class:

Locale

```
public static final class Locale.Builder
extends Object
```

`Builder` is used to build instances of `Locale` from values configured by the setters. Unlike the `Locale` constructors or `Locale.of()` factory methods, the `Builder` checks if a value configured by a setter satisfies the syntax requirements defined by the `Locale` class. A `Locale` object obtained from a `Builder` is well-formed and can be transformed to a well-formed IETF BCP 47 language tag without losing information.

Implementation Note:

The `Locale` class does not provide any syntactic restrictions on variant, while BCP 47 requires each variant subtag to be 5 to 8 alphanumerics or a single numeric followed by 3 alphanumerics. The method `setVariant` throws `IllformedLocaleException` for a variant that does not satisfy this restriction. If it is necessary to support such a variant, use `Locale.of(String, String, String)`. However, keep in mind that a `Locale` object obtained this way might lose the variant information when transformed to a BCP 47 language tag.

The following example shows how to obtain a `Locale` object using a `Builder`.

```
Locale aLocale = new Locale.Builder().setLanguage("sr").setScript("Latn").setRe
```

Builders can be reused; `clear()` resets all fields to their default values.

Since:

1.7

See Also:

`Locale.forLanguageTag(String)`,
`Locale.of(String, String, String)`

Constructor Summary

Constructors

Constructor	Description
`Builder()`	Constructs an empty Builder.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
Locale.Builder	**addUnicodeLocaleAttribut**(String attribute)	Adds a unicode locale attribute, if not already present, otherwise has no effect.
Locale	**build**()	Returns an instance of Locale obtained from the fields set on this builder.
Locale.Builder	**clear**()	Resets the builder to its initial, empty state.
Locale.Builder	**clearExtensions**()	Resets the extensions to their initial, empty state.
Locale.Builder	**removeUnicodeLocaleAttri**(String attribute)	Removes a unicode locale attribute, if present, otherwise has no effect.
Locale.Builder	**setExtension**(char key, String value)	Sets the extension for the given key.
Locale.Builder	**setLanguage**(String language)	Sets the language.
Locale.Builder	**setLanguageTag**(String languageTag)	Resets the Builder to match the provided IETF BCP 47 language tag.
Locale.Builder	**setLocale**(Locale locale)	Resets the Builder to match the provided locale.
Locale.Builder	**setRegion**(String region)	Sets the region.
Locale.Builder	**setScript**(String script)	Sets the script.
Locale.Builder		

	setUnicodeLocaleKeyword(String key, String type)	Sets the Unicode locale keyword type for the given key.
Locale.Builder	**setVariant**(String variant)	Sets the variant.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Builder

```
public Builder()
```

Constructs an empty Builder. The default value of all fields, extensions, and private use information is the empty string.

Method Details

setLocale

```
public Locale.Builder setLocale(Locale locale)
```

Resets the `Builder` to match the provided `locale`. Existing state is discarded.

All fields of the locale must be well-formed, see `Locale`.

Locales with any ill-formed fields cause `IllformedLocaleException` to be thrown, except for the following three cases which are accepted for compatibility reasons:

- Locale("ja", "JP", "JP") is treated as "ja-JP-u-ca-japanese"
- Locale("th", "TH", "TH") is treated as "th-TH-u-nu-thai"
- Locale("no", "NO", "NY") is treated as "nn-NO"

Parameters:

`locale` - the locale

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `locale` has any ill-formed fields.

NullPointerException - if locale is null.

setLanguageTag

public Locale.Builder setLanguageTag(String languageTag)

Resets the Builder to match the provided IETF BCP 47 language tag. Discards the
existing state. Null and the empty string cause the builder to be reset, like clear(). Legacy tags (see Locale.forLanguageTag(String)) are converted to their canonical form before being processed. Otherwise, the language tag must be well-formed (see Locale) or an exception is thrown (unlike Locale.forLanguageTag, which just discards ill-formed and following portions of the tag).

Duplicate variants are accepted and included by the builder. However, duplicate extension singleton keys and their associated type are accepted but ignored. The same behavior applies to duplicate locale keys and attributes within a U extension. Note that subsequent subtags after the occurrence of a duplicate are not ignored.

See converions for a full list of conversions that are performed on languageTag.

Parameters:

languageTag - the language tag

Returns:

This builder.

Throws:

IllformedLocaleException - if languageTag is ill-formed

See Also:

Locale.forLanguageTag(String)

setLanguage

public Locale.Builder setLanguage(String language)

Sets the language. If language is the empty string or null, the language in this Builder is removed. Otherwise, the language must be well-formed or an exception is thrown.

The typical language value is a two or three-letter language code as defined in ISO639.

Parameters:

language - the language

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `language` is ill-formed

setScript

```
public Locale.Builder setScript(String script)
```

Sets the script. If `script` is null or the empty string, the script in this `Builder` is removed. Otherwise, the script must be well-formed or an exception is thrown.

The typical script value is a four-letter script code as defined by ISO 15924.

Parameters:

`script` - the script

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `script` is ill-formed

setRegion

```
public Locale.Builder setRegion(String region)
```

Sets the region. If region is null or the empty string, the region in this `Builder` is removed. Otherwise, the region must be well-formed or an exception is thrown.

The typical region value is a two-letter ISO 3166 code or a three-digit UN M. 49 area code.

The country value in the `Locale` obtained from a `Builder` is always normalized to upper case.

Parameters:

`region` - the region

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `region` is ill-formed

setVariant

```
public Locale.Builder setVariant(String variant)
```

Sets the variant. If variant is null or the empty string, the variant in this `Builder` is removed. Otherwise, it must consist of one or more well-formed subtags, or an exception is thrown. Duplicate variants are accepted and included by the builder.

Note: This method checks if `variant` satisfies the IETF BCP 47 variant subtag's syntax requirements, and normalizes the value to lowercase letters. However, the `Locale` class does not impose any syntactic restriction on variant, and the variant value in `Locale` is case sensitive. To set such a variant, use `Locale.of(String, String, String)`.

Parameters:

`variant` - the variant

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `variant` is ill-formed

See Also:

`Locale.of(String, String, String)`

setExtension

```
public Locale.Builder setExtension(char key,
                                   String value)
```

Sets the extension for the given key. If the value is null or the empty string, the extension is removed. Otherwise, the extension must be well-formed or an exception is thrown.

Note: The key `UNICODE_LOCALE_EXTENSION` ('u') is used for the Unicode locale extension. Setting a value for this key replaces any existing Unicode locale key/type pairs with those defined in the extension. Duplicate locale attributes as well as locale keys and their associated type are accepted but ignored.

Note: The key `PRIVATE_USE_EXTENSION` ('x') is used for the private use code. To be well-formed, the value for this key needs only to have subtags of one to eight alphanumeric characters, not two to eight as in the general case.

Parameters:

`key` - the extension key

`value` - the extension value

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `key` is illegal or `value` is ill-formed

See Also:

`setUnicodeLocaleKeyword(String, String)`

setUnicodeLocaleKeyword

```
public Locale.Builder setUnicodeLocaleKeyword(String key,
                                              String type)
```

Sets the Unicode locale keyword type for the given key. If the type is null, the Unicode keyword is removed. Otherwise, the key must be non-null and both key and type must be well-formed or an exception is thrown.

Keys and types are converted to lower case.

Note:Setting the 'u' extension via `setExtension(char, String)` replaces all Unicode locale keywords with those defined in the extension.

Parameters:

`key` - the Unicode locale key

`type` - the Unicode locale type

Returns:

This builder.

Throws:

`IllformedLocaleException` - if `key` or `type` is ill-formed

`NullPointerException` - if `key` is null

See Also:

`setExtension(char, String)`

addUnicodeLocaleAttribute

```
public Locale.Builder addUnicodeLocaleAttribute(String attribute)
```

Adds a unicode locale attribute, if not already present, otherwise has no effect. The attribute must not be null and must be well-formed or an exception is thrown.

Parameters:

`attribute` - the attribute

Returns:

This builder.

Throws:

`NullPointerException` - if `attribute` is null

`IllformedLocaleException` - if `attribute` is ill-formed

See Also:

`setExtension(char, String)`

removeUnicodeLocaleAttribute

```
public Locale.Builder removeUnicodeLocaleAttribute(String attribute)
```

Removes a unicode locale attribute, if present, otherwise has no effect. The attribute must not be null and must be well-formed or an exception is thrown.

Attribute comparison for removal is case-insensitive.

Parameters:

`attribute` - the attribute

Returns:

This builder.

Throws:

`NullPointerException` - if `attribute` is null

`IllformedLocaleException` - if `attribute` is ill-formed

See Also:

`setExtension(char, String)`

clear

```
public Locale.Builder clear()
```

Resets the builder to its initial, empty state.

Returns:

This builder.

clearExtensions

```
public Locale.Builder clearExtensions()
```

Resets the extensions to their initial, empty state. Language, script, region and variant are unchanged.

Returns:

This builder.

See Also:

`setExtension(char, String)`

build

```
public Locale build()
```

Returns an instance of `Locale` obtained from the fields set on this builder.

This applies the conversions listed in `Locale.forLanguageTag(String)` when constructing a Locale. (Legacy tags are handled in `setLanguageTag(String)`.)

Returns:

A Locale.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.